

Logarithmischer Speicher & Weitere Klassen

Markus Hecher

Universität Potsdam, Germany

Vorlesung am July 20, 2026

Last updated: 2026-07-20

Zusammenfassung

- Definitionen C-harter und C-vollständiger Probleme für eine Klasse C.
- Verständnis für die Rolle vollständiger Probleme in der Komplexitätstheorie
- Sie sollten die Intuition von NP-Vollständigkeit verstanden haben (und NP-vollständige Probleme erkennen).
- Zwei fundamentale NP-vollständige Probleme: SAT und **3-SAT**.
- Viele Beispiele NP-vollständiger Probleme, z.B.: **CLIQUE**, **INDEPENDENT SET**, **VERTEX COVER**, **3-COLORABILITY**, **HAMILTON-PATH**, **HAMILTON-CYCLE**, **TSP(E)**, etc.
- Nützlichkeit von Reduktionen auf SAT.

Inhalt

- 1 Komplexität von SAT präzisiert
 - 2-SAT ist in P
 - 3-SAT ist NP-vollständig
- 2 Weitere Klassen
 - Logarithmischer Speicher
 - PSPACE (Polynomieller Speicher)
 - EXPTIME (Exponentialzeit)

Komplexität von SAT präzisiert

Varianten von Erfüllbarkeit

SAT

INSTANZ: Boolesche Formel φ .

FRAGE: Ist φ erfüllbar?

3-SAT

INSTANZ: Boolesche Formel φ in 3-CNF

FRAGE: Ist φ erfüllbar?

2-SAT

INSTANZ: Boolesche Formel φ in 2-CNF

FRAGE: Ist φ erfüllbar?

Komplexität von SAT präzisiert

2-SAT ist in P

Reduktion von 2-SAT auf Reachability

Es gibt eine Polynomialzeit-Reduktion von dem **2-SAT**-Problem auf das **REACHABILITY**-Problem.

Anmerkung

Wir haben bereits gesehen, dass das **REACHABILITY** Problem handhabbar ist (d.h., es kann in Polynomialzeit gelöst werden).

Folgerung: Das **2-SAT**-Problem ist handhabbar.

Reduktion von 2-SAT auf Reachability II

Beweis (Skizze).

Sei φ eine beliebige Instanz von **2-SAT**.

Wir definieren einen Graphen $G(\varphi)$ wie folgt:

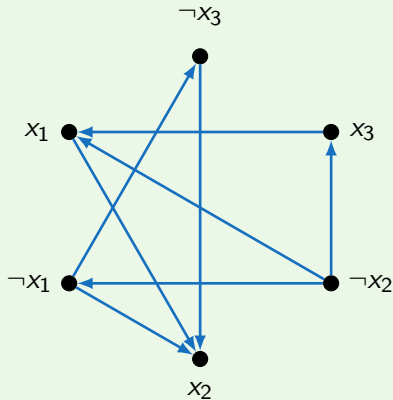
- Die Variablen von φ und deren Negationen formen die Knoten von $G(\varphi)$.
- Es gibt eine (gerichtete!) Kante (α, β) gdw. es eine Klausel $\bar{\alpha} \vee \beta$ oder $\beta \vee \bar{\alpha}$ in φ gibt, d.h., wenn (α, β) eine Kante ist, dann ist $(\bar{\beta}, \bar{\alpha})$ auch eine Kante ($\bar{\alpha}$ entspricht dem Komplement von α).
- **Semantik** der Kanten (α, β) : Wenn α *true* ist in einer erfüllbaren Belegung T von φ , dann muss β auch *true* sein in T .

Man kann zeigen, dass φ unerfüllbar ist, gdw. es eine Variable x und Pfade in $G(\varphi)$ gibt, nämlich von x nach $\neg x$ und von $\neg x$ nach x .

Beispiel.

$$\psi = (x_1 \vee x_2) \wedge (x_1 \vee \neg x_3) \wedge (\neg x_1 \vee x_2) \wedge (x_2 \vee x_3)$$

$\neg x_1 \Rightarrow x_2$
 $\neg x_2 \Rightarrow x_1$
 $\neg x_1 \Rightarrow \neg x_3$
 $x_3 \Rightarrow x_1$
 $x_1 \Rightarrow x_2$
 $\neg x_2 \Rightarrow \neg x_1$
 $\neg x_2 \Rightarrow x_3$
 $\neg x_3 \Rightarrow x_2$



Beweis (Skizze) fortgesetzt.

Wir erhalten damit folgenden Algorithmus für **2-SAT**:

- 1 Konstruiere den Graphen $G(\varphi)$ wie oben beschrieben;
- 2 Für alle Variablen x von φ : Prüfe, ob sowohl $\neg x$ von x im Graphen $G(\varphi)$ und x von $\neg x$ in $G(\varphi)$ erreichbar sind;
- 3 Wenn durch die REACHABILITY-Tests in Schritt 2, eine Variable x gefunden wird, sodass sowohl $\neg x$ von x als auch x von $\neg x$ erreicht werden, dann gibt der **2-SAT**-Algorithmus ein "Nein" zurück. Andernfalls returnieren er "Ja".

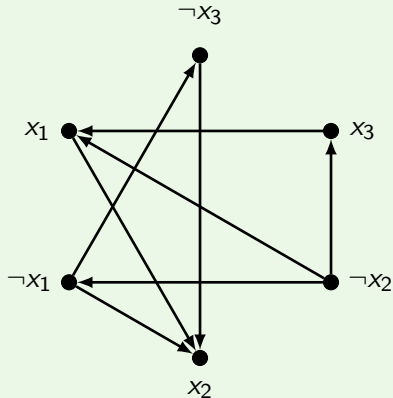


Beispiel (fortgesetzt).

$$\psi = (x_1 \vee x_2) \wedge (x_1 \vee \neg x_3) \wedge (\neg x_1 \vee x_2) \wedge (x_2 \vee x_3)$$

- x_1 erreicht nur x_2
- x_2 erreicht keinen anderen Knoten
- x_3 erreicht nur x_1 und x_2

⇒ ψ ist erfüllbar



Komplexität von SAT präzisiert

3-SAT ist NP-vollständig

Varianten von Erfüllbarkeit

- Wir wissen, dass **2-SAT** handhabbar (tractable) ist:
Man kann einen Polynomialzeit-Algorithmus zum Beispiel auch über eine Polynomialzeit-Reduktion auf **REACHABILITY** erhalten.
- Im Gegenzug sind SAT und **3-SAT** nicht handhabbar.

Das SAT-Problem ist DAS klassische NP-vollständige Problem. Es verbleibt NP-vollständig, selbst wenn wir die Boolesche Formel auf 3-CNF beschränken (**3-SAT**).

Komplexität von SAT und 3-SAT

Satz von Cook-Levin: SAT ist NP-vollständig.

Es gilt sogar die folgende stärkere Variante.

3-SAT ist NP-vollständig.

Beweis der NP-Mitgliedschaft.

SAT und auch **3-SAT** kann man durch den folgenden NP-Algorithmus nachweisen:

1. Rate eine Belegung T über die Variablen in φ .
2. Überprüfe, dass φ in T zu *true* auswertet.



Beweisidee der NP-Härte von SAT.

Um zu zeigen, dass SAT **als erstes Problem** NP-hart ist, müssen wir für **jedes** Problem $\mathcal{P}$ in NP zeigen, dass es eine Polynomialzeit-Reduktion von $\mathcal{P}$ auf SAT gibt.

Dadurch, dass NP unendlich viele Probleme hat, können wir nicht jedes Problem separat behandeln. Wir brauchen eine Methode, um alle Probleme in NP gemeinsam zu adressieren.

Die Idee ist wie folgt. Wir nehmen ein **beliebiges Problem** $\mathcal{P} \in \text{NP}$ und eine **beliebige Instanz** I von $\mathcal{P}$. Wir müssen zeigen, dass wir in Polynomialzeit eine Formel φ bauen können, sodass **I eine positive instanz von $\mathcal{P}$ ist $\Leftrightarrow \varphi$ ist erfüllbar**.

Weil $\mathcal{P} \in \text{NP}$, muss es per Definition eine polynomiell-balancierte und polynomiell-entscheidbare Zertifikatsrelation R für $\mathcal{P}$ geben.

Per Definition: I ist eine positive instanz von $\mathcal{P} \Leftrightarrow$ es gibt ein Objekt C von polynomieller Größe in $|I|$, sodass $(I, C) \in R$. Um $(I, C) \in R$ zu testen, haben wir einen Polynomialzeit-Algorithmus A zur Verfügung.

Beweisidee der NP-Härte von SAT II.

Für die Reduktion genügt es daher, in Polynomialzeit eine Formel φ zu konstruieren, sodass folgende Äquivalenz gilt:

φ ist erfüllbar $\Leftrightarrow$ es gibt ein polynomiell großes Objekt C , sodass Algorithmus A auf Eingabe (I, C) den Wert *true* liefert.

Die **gewünschte Boolesche Formel** φ muss 2 Aufgaben ausführen:

- 1 Die Kandidatszertifikate generieren,
- 2 Die Berechnung von A auf einem Kandidatspaar (I, C) simulieren.

Die 2. Aufgabe ist nicht trivial: Wie können wir eine Boolesche Formel verwenden, um Pseudocode zu simulieren? Wie behandeln wir **while**, **for**, **if/then/else**?

Antwort: Das kann man prinzipiell machen, aber es ist viel einfacher, stattdessen **Turingmaschinen** zu verwenden (und die Church-Turing These). Turingmaschinen sind ein **viel einfacheres Berechnungsmodell**, das aber immer noch stark genug ist im Hinblick der Berechenbarkeit und Komplexität. □

Komplexität von SAT und 3-SAT II

Beweisidee der NP-Härte von **3-SAT**.

Wir müssen von SAT auf **3-SAT** reduzieren, d.h.: Sei φ eine beliebige Boolesche Formel. Wir können zeigen, dass es eine Boolesche Formel $R(\varphi) = \psi$ gibt, sodass ψ in 3-CNF ist und φ erfüllbar ist $\Leftrightarrow \psi$ erfüllbar ist.

Anmerkung.

- Eine beliebige Boolesche Formel φ kann in **Polynomialzeit** in eine **erfüllungsäquivalente** Formel ψ in 3-CNF übersetzt werden.
- Im Allgemeinen sind φ und ψ **nicht logisch äquivalent** (wegen Hilfsvariablen!).
- Dieses Resultat ist nicht trivial: Die “einfache” Übersetzung nach CNF via den Gesetzen von den Regeln von de Morgan und Distributivität von $\wedge$ und $\vee$ führt üblicherweise zu einem exponentiellen Blow-up.

Beispiel: Die CNF-Formel, die **logisch äquivalent** zu $(x_1 \wedge y_1) \vee \dots \vee (x_n \wedge y_n)$ ist, ist exponentiell größer.

Weitere Klassen

Weitere Klassen

Logarithmischer Speicher

Die Klasse L (1)

L ist die Klasse der Probleme, die durch ein Programm mit logarithmischem Speicherplatzverbrauch gelöst werden können: $\mathcal{P} \in L$ wenn es ein Programm gibt, das für alle Instanzen I von $\mathcal{P}$ maximal $O(\log_2 |I|)$ Bits an Lese-/Schreibspeicher benötigt.

- Laufzeit ist eigentlich unbeschränkt! (aber polynomiell gebunden, später mehr).
 - Logarithmischer Platzverbrauch = sehr wenig Speicher, z.B. $\log_2(65536) = 16$.
 - Wenn wir $O(\log_2 |I|)$ umschreiben, haben wir ein Programm, das maximal $c \cdot \log_2 |I| + d$ Bits an Lese-/Schreibspeicher verwendet (mit Konstanten c, d).
- ↪ Also ist dieser Speicher eines Programmes beschränkt auf **konstant** viele
- **Pointer** (Zeiger, adressieren Speicher),
 - **Zähler**, und
 - **Boolesche Flags**.

Die Klasse L (2)

- Wir müssen annehmen, dass die Eingabe für Programme mit logarithmischem Speicherplatzbedarf in Nur-Lesespeicher (read-only) gespeichert wird!
 - Weil: $c \cdot \log_2 n + d < n$ für hinreichend große n .
 - Ohne dieser Annahme würde das Programm auf großem Input nicht im Stande sein, ihren Input zu traversieren.
- Beispielproblem in L:

FIND-NODE

INSTANZ: Zahl $n \in \mathbb{N}$ und ein Baum T wo jeder Knoten mit einer natürlichen Zahl gelabelt wird.

FRAGE: Enthält T einen Knoten der mit n gelabelt wird?

↪ **FIND-NODE** $\in L$, weil wir T mit Tiefensuche (DFS) mit nur drei Zeiger auf Knoten in T traversieren können: current, next, und aux.

FIND-NODE mit logarithmischem Platzbedarf

Wir verwenden folgende Methoden (implementierbar mit logarithmischem Platzbedarf):

- $root(T)$ gibt die Wurzel von T zurück
 - $root(T) = nil$ wenn T leer ist
- $firstChild(T, x)$ gibt das erste Kind eines Knoten x in T zurück
 - $firstChild(T, x) = nil$ wenn so ein Kind nicht existiert
- $rightSibling(T, x)$ retourniert rechten Geschwisterknoten eines Knoten x in T
 - $rightSibling(T, x) = nil$ wenn so ein Geschwisterknoten nicht existiert
- $parent(T, x)$ gibt den Elternknoten eines Knoten x in T zurück
 - $parent(T, x) = nil$ wenn x die Wurzel in T ist
- $isChildOf(T, x_1, x_2)$ gibt **true** zurück gdw. x_1 ist ein Kind von x_2 in T
- $isLeaf(T, x)$ gibt **true** zurück gdw. $firstChild(T, x) = nil$
- $hasRightSibling(T, x)$ liefert **true** gdw. $rightSibling(T, x) \neq nil$
- $labeling(T, x)$ liefert das Label von x in T

// Speicherplatz-effiziente Implementierung von Tiefensuche (DFS)

Boolean *find*(*Tree T*, *Integer val*)*current* = *root*(*T*);**if** *labeling*(*T*, *current*) = *val* **then return true**;*next* = *firstChild*(*T*, *current*);**while** (*next* != *nil*) {**if** *isChildOf*(*T*, *next*, *current*) **and** *!isLeaf*(*T*, *next*) **then** {*current* := *next*; *next* := *firstChild*(*T*, *next*) }**else if** *isChildOf*(*T*, *next*, *current*) **and** *isLeaf*(*T*, *next*) **then** {*aux* := *current*; *current* := *next*; *next* := *aux* }**else if** *isChildOf*(*T*, *current*, *next*) **and** *hasRightSibling*(*T*, *current*) **then**{ *aux* := *current*; *current* := *next*; *next* := *rightSibling*(*T*, *aux*) }**else if** *isChildOf*(*T*, *current*, *next*) **and** *!hasRightSibling*(*T*, *current*) {*current* := *next*; *next* := *parent*(*T*, *next*) }**if** *labeling*(*T*, *current*) = *val* **then return true**

}

return false

L und P

Theorem 1.

$L \subseteq P$, d.h. jedes Problem, das mit logarithmischem Speicherplatzbedarf gelöst werden kann, kann auch in Polynomialzeit gelöst werden.

Beweis (Skizze).

Wir wissen, dass ein log-Speicher Program Π_1 auf Input I maximal $c \cdot \log_2 |I| + d$ Bits an Lese-/Schreibspeicher benötigt (inkludiert Programmzähler, Register, etc.), wobei c, d Konstanten sind. Beachte: Der Zustand der Ausführung von Π_1 auf I ist **eindeutig beschrieben durch den Speicherinhalt**. Wir haben $2^{c \cdot \log_2 |I| + d}$ mögliche Konfigurationen, in denen sich Π_1 befinden kann. Wir folgern $2^{c \cdot \log_2 |I| + d} = (2^{\log_2 |I|})^c \cdot 2^d = |I|^c \cdot 2^d$; weil c, d Konstanten sind, sind es **polynomiell verschiedene Konfigurationen**.

Basierend auf obigen Beobachtungen können wir ein Polynomialzeit-Programm Π_2 basteln, das **alle möglichen Konfigurationen** von Π_1 auf I listet und entscheidet, ob Π_1 dann *true* oder *false* auf I liefert. □

Weitere Klassen

PSPACE (Polynomieller Speicher)

Die Klasse PSPACE

PSPACE ist die Klasse aller Probleme die man durch ein Programm mit polynomiellen Speicher lösen kann, d.h., $\mathcal{P} \in \text{PSPACE}$ wenn es ein Programm Π gibt, sodass: für alle Instanzen I von $\mathcal{P}$ verwendet Π max. $O(|I|^k)$ Bits an Speicher für Konstante k .

- Die Klasse ist sehr breit und mächtig, denn viele Probleme des realen Lebens fallen in diese Klasse.
 - “polynomieller Speicher” meint, dass die Programmausführung in exponentiell vielen unterschiedlichen Zuständen sein kann (vs. polynomiell viele Zustände bei L)
- ↪ PSPACE-vollständige Probleme sind nicht handhabbar (wir werden sehen, dass $\text{NP} \subseteq \text{PSPACE}$).

Ein Problem in PSPACE: Tic-Tac-Toe

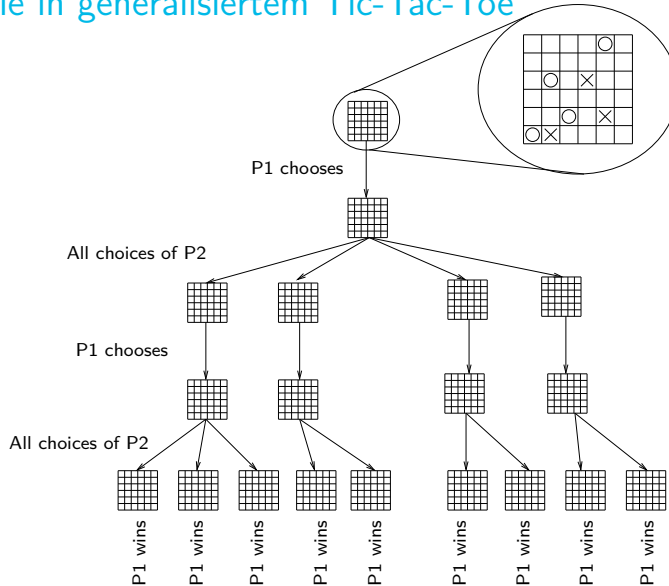
- (n, k) -Tic-Tac-Toe ist eine Generalisierung Tic-Tac-Toe, gespielt auf $n \times n$ Brett.
 - Der 1. Spieler, der eine Reihe mit k seiner Tokens erhält, ist der Gewinner.
- ↪ Das Spiel endet nach maximal n^2 vielen Schritten.

TTT-WINNING-STRATEGY

INSTANZ: Zahlen $n, k \in \mathbb{N}$ und Konfiguration C eines $n \times n$ Bretts.

FRAGE: Hat Spieler 1 eine Gewinnstrategie für das (n, k) -Tic-Tac-Toe Spiel, das mit Konfiguration C startet?

Gewinnstrategie in generalisiertem Tic-Tac-Toe



PSPACE: Zusammenfassung

TTT-WINNING-STRATEGY

Problem ist in PSPACE (und ist PSPACE-complete):

- es genügt, einen Stack mit maximal n^2 Konfigurationen zu haben;
- jede Konfiguration hat polynomiellen Speicherplatzbedarf.

Beobachtungen:

- **Alternierung** ist typisch für PSPACE-vollständige Probleme, d.h.:
es gibt einen Zug für Spieler P1, sodass
für alle Züge von Spieler P2,
es gibt einen Zug für Spieler P1, ...
- Probleme in PSPACE können durch polynomiell-tiefe **verschachtelte (nested) Schleifen gelöst werden**: Für TTT-WINNING-STRATEGY ist die Verschachtelungstiefe durch n^2 gebunden.

Andere Probleme in PSPACE

First-order Logik

Prädikatenlogik (Logik 1. Stufe) über eine algebraische Struktur.

SQL Auswertung

Auswertung einer SQL Abfrage über eine Datenbank.

Beide Probleme sind quasi ident!

Auswertung Quantifizierter Boolescher Formeln

Boolesche Formel mit Quantoren der Form $\exists X_1. \forall X_2. \dots \exists X_n. \varphi(X_1, X_2, \dots, X_n)$

$\rightsquigarrow$ Viele Spiele (2-Player Games) fallen in PSPACE.

PSPACE vs. P

Theorem 2.

$P \subseteq PSPACE$, d.h., jedes Problem, das mit polynomielltem Speicherplatz gelöst werden kann, kann auch in Polynomialzeit gelöst werden.

Beweis (Skizze).

Trivial; jedes Programm mit polynomieller Laufzeit verwendet nie mehr als polynomiellen Speicherplatz, d.h., wenn Π ein Polynomialzeit-Programm ist, um zu entscheiden ob $\mathcal{P} \in P$, dann läuft Π auch mit polynomiellen Speicherplatzverbrauch, daher $\mathcal{P} \in PSPACE$. □

PSPACE vs. NP

Theorem 3.

$NP \subseteq PSPACE$.

Beweis (Skizze).

Sei Problem $\mathcal{P} \in NP$. Dann gibt es eine Zertifikatsrelation R für $\mathcal{P}$, die polynomiell-balanciert und polynomiell-entscheidbar ist.

Wir können ein Programm designen, das Instanzen I von $\mathcal{P}$ mit polynomielltem Speicherverbrauch löst:

- Iteriere über alle Kandidatzertifikate (eines nach dem anderen), behalte nur eines im Speicher. Weil R polynomiell-balanciert ist, genügt es, uns auf Kandidaten mit polynomieller Größe zu fokussieren.
- Für jeden Kandidaten C : Prüfe ob $(I, C) \in R$. Weil R polynomiell-entscheidbar ist, braucht jeder Test maximal Polynomialzeit, daher polynomiellen Speicherplatz.



Weitere Klassen

EXPTIME (Exponentialzeit)

Die Klasse EXPTIME

EXPTIME ist die Klasse der Probleme, die in **Exponentialzeit** lösbar sind, d.h., $\mathcal{P} \in \text{EXPTIME}$, wenn es ein Programm gibt, das alle Instanzen I von $\mathcal{P}$ in Zeit $O(2^{|I|^k})$ für Konstante k löst.

Folgende Inklusionen sind bekannt:

- $\text{PSPACE} \subseteq \text{EXPTIME}$ (selbes Argument wie für $L \subseteq P$)
- $P \subsetneq \text{EXPTIME}$ (nichttriviales Argument via Diagonalisierung)

Beispielprobleme in EXPTIME:

- Existenz einer Gewinnstrategie in **GO** (generalisiert auf $n \times n$ Spielbrett); Grund: es gibt keine polynomielle Schranke der Spiellänge,
- Auswertung von DATALOG Abfragen (SQL erweitert um Rekursion).

Diskussion

Es gibt sehr viele weitere Komplexitätsklassen... zum Beispiel:

- 2-EXPTIME = Probleme lösbar in $O(2^{2^{|I|^k}})$ Zeit
- 3-EXPTIME = Probleme lösbar in $O(2^{2^{2^{|I|^k}}})$ Zeit
- ...
- EXPSPACE = Probleme lösbar mit $O(2^{|I|^k})$ Speicherplatz
- 2-EXPSPACE = Probleme lösbar mit $O(2^{2^{|I|^k}})$ Speicherplatz
- 3-EXPSPACE = Probleme lösbar mit $O(2^{2^{2^{|I|^k}}})$ Speicherplatz
- ...